

Genetic Algorithm–Based DNN Classifier
Architecture Optimization
for Multi-Sensor Vibration Diagnosis

Standalone Methodological Chapter

(Three-Section Structure; separate from the main thesis PDF)

Contents

1	GA-Based DNN Classifier Architecture Optimization	5
1.1	DNN Architecture Representation and Search Space	5
1.1.1	Motivation: Why Architecture Must Be Searched	5
1.1.2	Chromosome Encoding	6
1.1.3	Search Space Definition	6
1.1.4	Cardinality: Why Exhaustion Is Impossible	7
1.1.5	Parameter-Count Constraint and Repair	7
1.1.6	Decoding a Chromosome into an MLP	8
1.1.7	Section 1 Summary	8
1.2	GA-Based Architecture Optimization Procedure	9
1.2.1	Why a Genetic Algorithm	9
1.2.2	Step A — Initialize the Population	10
1.2.3	Step B — Fitness Evaluation (Macro-F1)	11
1.2.4	Step C — Tournament Selection	12
1.2.5	Step D — Crossover and Mutation	13
1.2.6	Step E — Elitism and Replacement	14
1.2.7	Step F — Termination and Best Architecture	14
1.2.8	Worked Numerical Crossover Trace	15
1.2.9	Main GA Loop	15
1.2.10	Section 2 Summary	16
1.3	Configuration-Specific Dynamic Architecture Discovery	16
1.3.1	One Configuration at a Time	16
1.3.2	Operational Protocol for a New Configuration	17

1.3.3	Three Deployment Policies Compared	17
1.3.4	State Variables During a Run	17
1.3.5	What This Method Deliberately Does Not Do	18
1.3.6	Common Implementation Pitfalls	18
1.3.7	Discussion	18
1.3.8	Chapter Summary	19
A	Notation and Quick Reference	20
A.1	Reproducibility Checklist	20
A.2	Formula–Reason Index	21

List of Figures

1.1	Chromosome encoding of one candidate DNN classifier.	6
1.2	Deterministic decoding of a chromosome into an MLP classifier.	8
1.3	Workflow of the adopted GA architecture optimizer.	9
1.4	Six procedural steps of the GA search (encoding already fixed in Section 1).	10
1.5	Initialization flow with parameter-budget repair.	10
1.6	Fitness evaluation pipeline for one individual.	11
1.7	Illustrative GA convergence: best and mean validation macro-F1 versus generation.	12
1.8	Tournament selection diagram.	12
1.9	Single-point crossover producing a child from two parents.	13
1.10	Configuration-specific architecture discovery with the same GA operators.	16
1.11	End-to-end path from fused features to a deployed configuration-specific architecture.	18

List of Tables

1.1	Architecture search space and reasons for the allowed values.	7
1.2	Classical GA stages mapped to this section's procedure.	9
1.3	Atomic mutation types.	14
1.4	GA hyperparameters and reasons.	15
A.1	Notation.	20
A.2	Core formulas and why they were chosen.	21

Chapter 1

GA-Based DNN Classifier Architecture Optimization

This standalone chapter presents the genetic algorithm (GA) method used to discover deep neural network (DNN) classifier architectures for multi-sensor vibration diagnosis. The classifier input is the fused feature vector $\mathbf{Z}_{fusion} \in \mathbb{R}^{448}$. The chapter is organized in three sections: (1) how candidate architectures are represented and what search space is used; (2) how the GA searches that space with selection, crossover, mutation, and elitism; (3) how the same procedure is applied one machine/sensor configuration at a time to obtain $\mathbf{C}^$. Every formula is accompanied by the reason it was chosen. This document does not modify the main doctoral thesis files.*

1.1 DNN Architecture Representation and Search Space

1.1.1 Motivation: Why Architecture Must Be Searched

Manual design of DNN classifiers for fault diagnosis is brittle. A network that works for one sensor layout may fail for another because the decision boundary induced by $\mathbf{Z}_{fusion}$ changes with the sensing configuration. Exhaustive trial of depths, widths, activations, and dropout rates is combinatorial: even a modest discrete grid already contains enormous numbers of candidates when variable depth is allowed, and each candidate requires a full

training run. Section 1.1 therefore defines *what* is searched: a readable chromosome, a discrete search space, and hard feasibility constraints. Section 1.2 defines *how* it is searched; Section 1.3 defines *how the result is used* per configuration.

1.1.2 Chromosome Encoding

A candidate multilayer perceptron is encoded as

$$\mathbf{C} = [L, N_1, \dots, N_L, A_1, \dots, A_L, D_1, \dots, D_L] \quad (1.1)$$

where L is the number of hidden layers, N_i the width of layer i , A_i its activation, and D_i its dropout rate. The input width is fixed at 448 ($\mathbf{Z}_{fusion}$) and the output width at $K = 8$ (machine conditions). Those ends are *not* evolved.

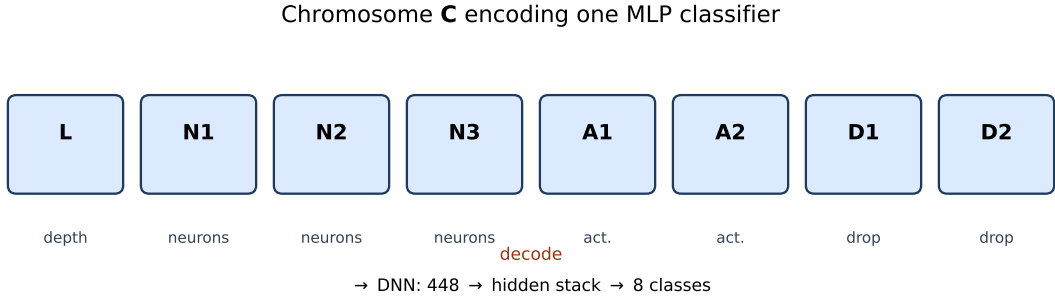


Figure 1.1: Chromosome encoding of one candidate DNN classifier.

Why this encoding? Every gene is something a vibration engineer already understands: how deep, how wide, which nonlinearity, how much regularization. When the GA returns $\mathbf{C}^*$, one can write down the winning MLP without specialized NAS tooling.

1.1.3 Search Space Definition

$$\begin{aligned} L &\in \{1, 2, 3, 4, 5\}, \\ N_i &\in \{32, 64, 128, 256, 512\}, \\ A_i &\in \{\text{ReLU}, \text{Tanh}, \text{LeakyReLU}\}, \\ D_i &\in \{0, 0.1, 0.2, 0.3, 0.5\}. \end{aligned} \quad (1.2)$$

Table 1.1: Architecture search space and reasons for the allowed values.

Gene	Allowed values	Why
L	$\{1, \dots, 5\}$	Compact MLPs; industrial latency
N_i	$\{32, 64, 128, 256, 512\}$	GPU-friendly widths; enough capacity for 448-D features
A_i	ReLU, Tanh, LeakyReLU	Standard activations that train stably with Adam
D_i	$\{0, 0.1, 0.2, 0.3, 0.5\}$	From no dropout to strong regularization
Input / out-put	448 / 8	Fixed by $\mathbf{Z}_{fusion}$ and 8 machine conditions

Why these discrete sets? They cover the practical range of compact MLPs on 448-D fused features without admitting pathologically wide nets.

1.1.4 Cardinality: Why Exhaustion Is Impossible

For fixed L , each layer has $5 \times 3 \times 5 = 75$ combinations, so

$$|\mathcal{S}| = \sum_{L=1}^5 75^L \approx 2.4 \times 10^9. \quad (1.3)$$

Why quote this formula? It justifies population-based stochastic search. Enumeration is impossible; a GA is not an over-engineered replacement for “try three networks by hand.”

1.1.5 Parameter-Count Constraint and Repair

With $w_0 = 448$, $w_{L+1} = 8$, $w_i = N_i$,

$$P(\mathbf{C}) = \sum_{i=0}^L w_{i+1}(w_i + 1) \leq 2 \times 10^6. \quad (1.4)$$

Why a hard parameter cap? Fitness evaluation trains each candidate fully. Unbounded widths would dominate GPU budget and bias the search toward huge overfit models.

Worked example. For $L = 3$, $(N_1, N_2, N_3) = (256, 128, 64)$,

$$P = 256 \cdot 449 + 128 \cdot 257 + 64 \cdot 129 + 8 \cdot 65 = 156616,$$

well under the budget. A five-layer stack of width 512 often exceeds 2×10^6 and is repaired.

Repair rule. While $P(\mathbf{C}) > 2 \times 10^6$, reduce the largest N_i to the next-smaller allowed width; if already minimal and $L > 1$, delete a layer; else resample. Repair is preferred to pure rejection because it preserves crossover events.

1.1.6 Decoding a Chromosome into an MLP

Decoding is deterministic: read L ; stack L dense layers with stored (N_i, A_i, D_i) ; append a softmax output of size 8.

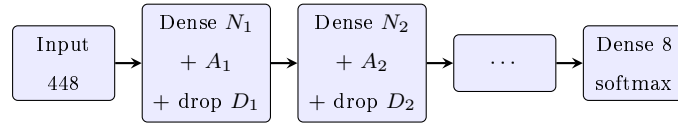


Figure 1.2: Deterministic decoding of a chromosome into an MLP classifier.

Why deterministic decoding? Randomness belongs in GA operators and weight initialization during fitness training, not in the mapping from genotype to architecture. Reproducibility requires a one-to-one decode.

1.1.7 Section 1 Summary

Section 1.1 fixed the searchable object: chromosome (1.1), space (1.2), cardinality (1.3), and budget (1.4). The next section searches that object with a genetic algorithm.

1.2 GA-Based Architecture Optimization Procedure

1.2.1 Why a Genetic Algorithm

A GA maintains a population of candidates, evaluates fitness, selects parents, creates children by crossover/mutation, and iterates over generations. It does *not* need a differentiable objective. That property is essential: fitness is validation macro-F1 after training—a black-box scalar.

Table 1.2: Classical GA stages mapped to this section’s procedure.

Classical stage	This chapter	Formula / rule
Encoding	Section 1.1	Chromosome (1.1)
Initialization	Step A below	Population (1.5)
Evaluation	Step B	Fitness (1.6)
Selection	Step C	Tournament $t = 3$
Variation	Step D	Crossover (1.11), mutation
Replacement	Step E	Elitism $E = 2$
Termination	Step F	$G_{\max}$ / stagnation; best (1.12)

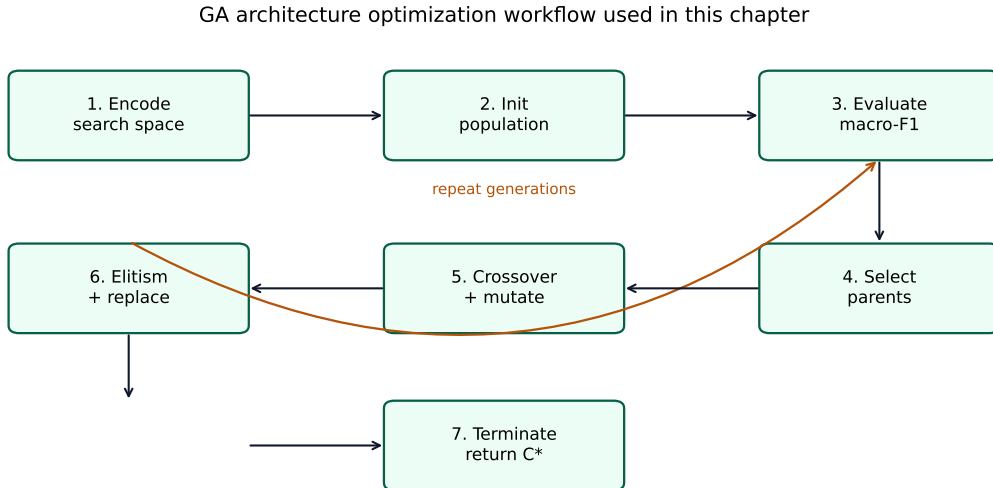


Figure 1.3: Workflow of the adopted GA architecture optimizer.

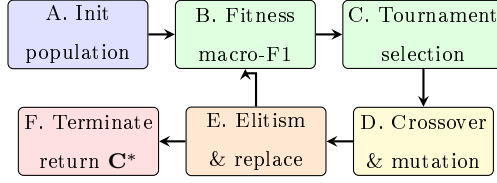


Figure 1.4: Six procedural steps of the GA search (encoding already fixed in Section 1).

Why GA rather than grid search or continuous NAS? Grid search scales exponentially in discrete genes. Continuous NAS relaxations do not match inherently categorical choices (depth, activation type, dropout). A classical chromosome matches the decision variables directly.

1.2.2 Step A — Initialize the Population

Draw $N = 30$ chromosomes by uniform sampling from Equation (1.2), then repair:

$$\mathcal{P}_0 = \{\mathbf{C}_1, \dots, \mathbf{C}_{30}\}. \quad (1.5)$$

Why $N = 30$? Balances diversity against evaluation cost (each individual needs a training run).

Why uniform random init? Maximizes initial coverage. Hand-seeded anchors can speed progress but bias comparisons; the baseline uses pure random init. After init, log histograms of L and widths; redraw if the draw is pathological (e.g., all $L = 1$).

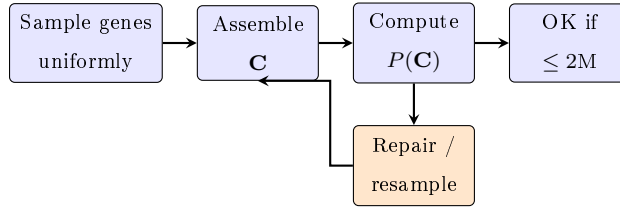


Figure 1.5: Initialization flow with parameter-budget repair.

1.2.3 Step B — Fitness Evaluation (Macro-F1)

Decode $\mathbf{C}_i$, train on the training split, and set

$$\text{Fitness}(\mathbf{C}_i) = \text{Macro-F1}(\mathbf{C}_i) = \frac{1}{K} \sum_{k=1}^K \frac{2 \text{Prec}_k \text{Rec}_k}{\text{Prec}_k + \text{Rec}_k} \quad (1.6)$$

on the validation set. Failed trainings score 0.

Why macro-F1 instead of accuracy? Accuracy can be dominated by the healthy class. Macro-F1 weights every machine condition equally—appropriate for safety-critical diagnosis.

Why validation, not test? Test data stay untouched until $\mathbf{C}^*$ is frozen.

For class k ,

$$\text{Prec}_k = \frac{TP_k}{TP_k + FP_k + \varepsilon}, \quad \text{Rec}_k = \frac{TP_k}{TP_k + FN_k + \varepsilon}. \quad (1.7)$$

Why not use cross-entropy as fitness? Cross-entropy trains weights inside a fixed net; it is not the operational class-balanced metric reported to engineers. Macro-F1 aligns search with the claimed performance measure.

Fixed training recipe (not evolved). Adam, learning rate 10^{-3} , batch size 64, max 50 epochs, early stopping patience 7, weight seed derived from a hash of $\mathbf{C}_i$. Learning-rate genes are deliberately excluded so fitness isolates architecture.

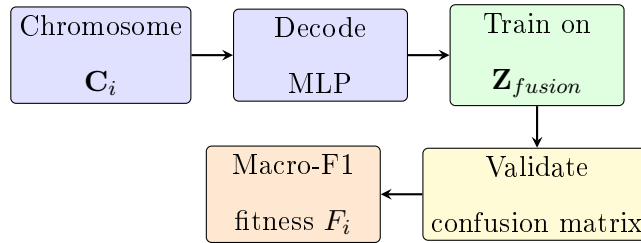


Figure 1.6: Fitness evaluation pipeline for one individual.

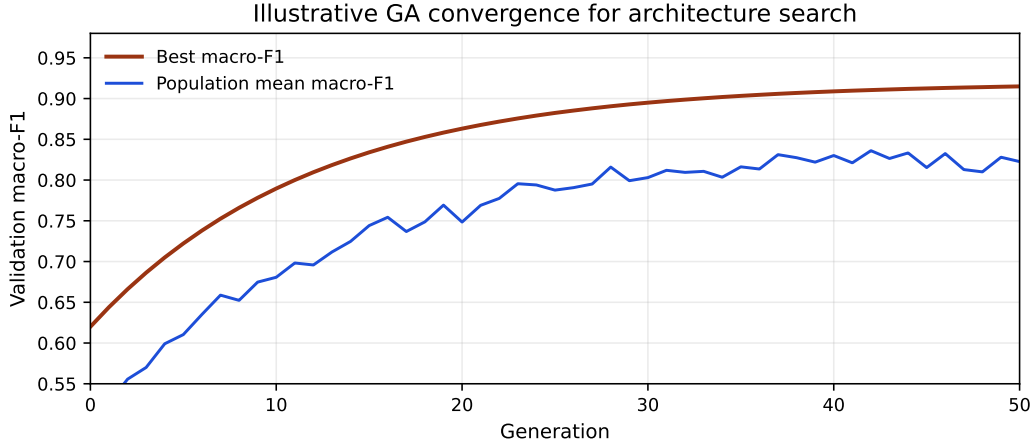


Figure 1.7: Illustrative GA convergence: best and mean validation macro-F1 versus generation.

1.2.4 Step C — Tournament Selection

Tournament size $t = 3$: sample three individuals without replacement; return the highest fitness. Two tournaments yield two parents.

Why $t = 3$? Moderate selection pressure: the global best often wins, but weaker parents still breed, preserving depth diversity.

The probability that the unique best enters a given tournament is

$$\frac{t}{N} = \frac{3}{30} = 0.1. \quad (1.8)$$

Why not roulette-wheel selection? Roulette needs fitness scaling when scores cluster near 0.9. Tournament needs none.

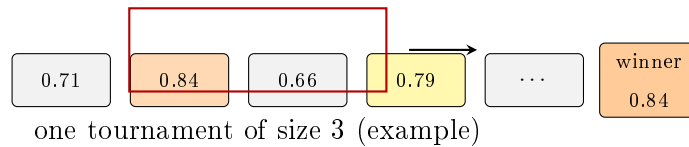


Figure 1.8: Tournament selection diagram.

1.2.5 Step D — Crossover and Mutation

With probability $p_c = 0.8$, build a child from parents $\mathbf{C}^{(p)}$ and $\mathbf{C}^{(q)}$:

$$L^{(c)} \leftarrow L^{(p)} \text{ or } L^{(q)} \text{ with probability } \frac{1}{2}, \quad (1.9)$$

$$r \sim \text{Uniform}\{1, \dots, L^{(c)}\}, \quad (1.10)$$

$$\text{layer}(\mathbf{C}^{(c)}, i) \leftarrow \begin{cases} \text{layer}(\mathbf{C}^{(p)}, \pi(i; L^{(p)})) & i < r, \\ \text{layer}(\mathbf{C}^{(q)}, \pi(i; L^{(q)})) & i \geq r, \end{cases} \quad (1.11)$$

where $\text{layer}(\mathbf{C}, i) = (N_i, A_i, D_i)$ and $\pi(i; L) = ((i - 1) \bmod L) + 1$.

Why single-point crossover? Preserves contiguous layer blocks that already work together.

Why wrap indexing? Chromosomes have variable length; wrapping lets a shallow parent still donate genes to a deeper child.

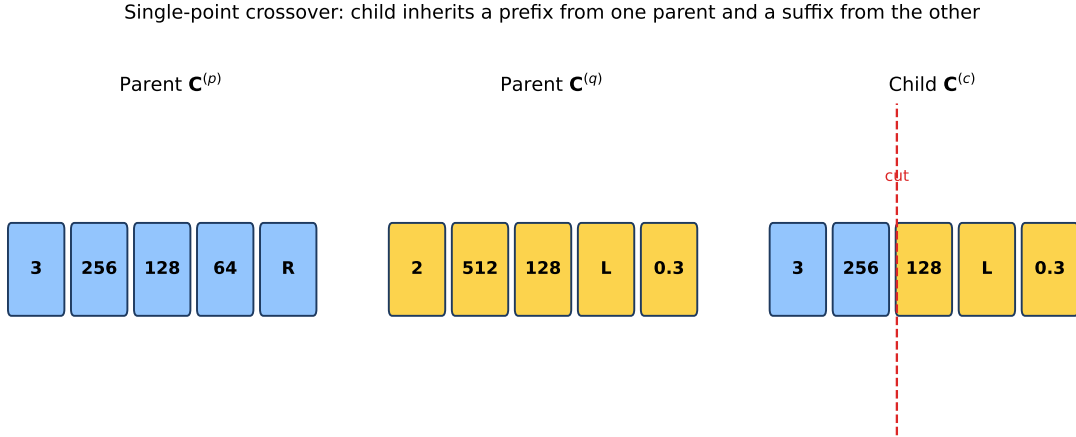


Figure 1.9: Single-point crossover producing a child from two parents.

Why $p_c = 0.8$? Strong recombination; the remaining 0.2 copies a parent unchanged so good genotypes are not always disrupted.

Mutation. With probability $p_m = 0.15$, change exactly one gene to a different legal value; repair afterward.

Table 1.3: Atomic mutation types.

Mutation	Effect / reason
Increase L	Insert a hidden layer (explore deeper boundaries)
Decrease L	Remove last layer (explore shallower models)
Change N_i	Widen/narrow capacity locally
Change A_i	Swap nonlinearity
Change D_i	Adjust regularization

Why one-gene mutation at rate 0.15? Local edits protect strong architectures while still enabling escape from local optima.

1.2.6 Step E — Elitism and Replacement

Copy the top $E = 2$ individuals unchanged into the next generation; fill remaining slots with children.

Why elitism of 2? The best validation macro-F1 must not be lost to unlucky mutation or noisy training.

1.2.7 Step F — Termination and Best Architecture

Stop at $G_{\max} = 50$ generations or when best fitness improves by less than $\varepsilon = 0.001$ for five consecutive generations. Return

$$\mathbf{C}^* = \arg \max_{\mathbf{C}_i \in \mathcal{P}} \text{Fitness}(\mathbf{C}_i). \quad (1.12)$$

Why these stopping rules? A hard cap bounds compute; stagnation saves budget when the population plateaus.

Table 1.4: GA hyperparameters and reasons.

Parameter	Value	Why
Population N	30	Diversity vs. training cost
Generations $G_{\max}$	50	Enough to reach plateau on this space
Crossover p_c	0.8	Strong recombination of layer blocks
Mutation p_m	0.15	Local exploration without destroying elites
Tournament t	3	Moderate selection pressure
Elitism E	2	Preserve best architectures
Fitness	Macro-F1	Class-balanced diagnostic goal
Param. budget	2×10^6	Tractable evaluations
Stagnation	5 gens / 0.001	Early stop when progress stalls

1.2.8 Worked Numerical Crossover Trace

Parents:

$$\mathbf{C}^{(p)} = [3, 256, 128, 64, \text{ReLU}, \text{ReLU}, \text{Tanh}, 0.1, 0.2, 0],$$

$$\mathbf{C}^{(q)} = [2, 512, 128, \text{ReLU}, \text{LeakyReLU}, 0.3, 0.1].$$

If $L^{(c)} = 3$ and cut $r = 2$, one legal child is

$$\mathbf{C}^{(c)} = [3, 256, 128, 512, \text{ReLU}, \text{LeakyReLU}, \text{ReLU}, 0.1, 0.1, 0.3],$$

subject to repair if $P(\mathbf{C}^{(c)})$ exceeds the budget. Variable-length crossover is the step most often under-specified; this trace is normative.

1.2.9 Main GA Loop

1. Sample and repair $\mathcal{P}_0$.
2. While $g < G_{\max}$ and not stagnated: evaluate fitness; keep elites; fill children by tournament $\rightarrow$ crossover $\rightarrow$ mutation $\rightarrow$ repair; update stagnation.

3. Return $\mathbf{C}^*$.

Complexity is dominated by

$$\mathcal{O}(N \cdot G \cdot T), \quad (1.13)$$

where T is the cost of one candidate training. The GA is an offline design-time tool.

1.2.10 Section 2 Summary

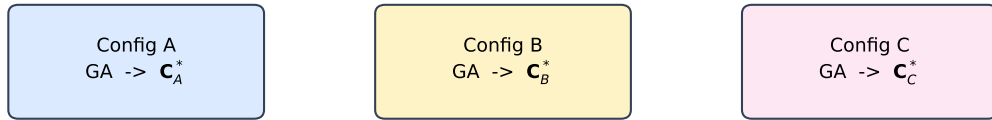
Section 1.2 specified the full search procedure from population initialization to $\mathbf{C}^*$, with explicit reasons for every operator and hyperparameter. The next section applies that procedure configuration by configuration.

1.3 Configuration-Specific Dynamic Architecture Discovery

1.3.1 One Configuration at a Time

The contribution of this chapter is not only the GA operators, but their *configuration-specific* use. The same Section 1.2 procedure is re-run for each machine/sensor setup:

$$\mathbf{X}_A \xrightarrow{\text{GA}} \mathbf{C}_A^*, \quad \mathbf{X}_B \xrightarrow{\text{GA}} \mathbf{C}_B^*, \quad \mathbf{X}_C \xrightarrow{\text{GA}} \mathbf{C}_C^*. \quad (1.14)$$



Same GA operators and hyperparameters; different data $\Rightarrow$ possibly different architectures

One-configuration-at-a-time optimization

Figure 1.10: Configuration-specific architecture discovery with the same GA operators.

The scientific claim is that optimal architectures need not coincide:

$$\exists \mathbf{X}_A, \mathbf{X}_B : \quad \mathbf{C}_A^* \neq \mathbf{C}_B^* \quad \text{and} \quad \text{Macro-F1}(\mathbf{C}_A^*; \mathbf{X}_A) > \text{Macro-F1}(\mathbf{C}_B^*; \mathbf{X}_A). \quad (1.15)$$

Why re-search rather than one universal net? Sensing layouts change the distribution of $\mathbf{Z}_{fusion}$. Transferring $\mathbf{C}_A^*$ to configuration B without re-search is a valid baseline, but this method treats architecture as configuration-dependent and provides a repeatable way to obtain it.

1.3.2 Operational Protocol for a New Configuration

1. Collect multi-sensor data for the new configuration.
2. Build $\mathbf{Z}_{fusion}$ features and labels; split train/validation/test.
3. Run the GA of Section 1.2 with Table 1.4.
4. Deploy the decoded MLP of $\mathbf{C}^*$ as the online classifier.

1.3.3 Three Deployment Policies Compared

1. **Universal fixed DNN:** one hand-designed architecture for all configurations.
2. **Transfer:** run GA on A , reuse $\mathbf{C}_A^*$ on B without re-search.
3. **Re-search (this chapter):** run GA independently on each configuration.

Why present all three? Policy 3 is the methodological recommendation. Policies 1–2 are necessary experimental baselines: if transfer already matches re-search, the plant can skip compute. The chapter does not assume re-search always wins by a large margin; it provides claim (1.15) and the protocol to test it.

1.3.4 State Variables During a Run

Maintain: population $\mathcal{P}_g$; fitness $\{F_i\}$; elite set of size E ; historical best F_{best} ; stagnation counter; optional archive of evaluated chromosomes. Given the same splits, seeds, and hyperparameters, two implementations should produce statistically consistent $\mathbf{C}^*$ distributions.

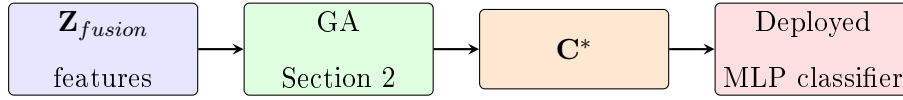


Figure 1.11: End-to-end path from fused features to a deployed configuration-specific architecture.

1.3.5 What This Method Deliberately Does Not Do

- Evolve learning rates or batch sizes (would confound architecture effects).
- Return a Pareto front of accuracy vs. size (hard cap + single macro-F1 objective).
- Use one-shot weight-sharing NAS (prefers full training per candidate).
- Co-evolve weights with topology (backprop trains weights for each decoded MLP).

1.3.6 Common Implementation Pitfalls

- Using test macro-F1 inside the GA (contaminates selection).
- Forgetting wrap indexing in crossover (illegal children when depths differ).
- Evolving learning rate with architecture (confounds structure claims).
- Random splits of temporally adjacent samples (inflated fitness).
- No elitism (best architectures disappear under noise).

1.3.7 Discussion

The fitness landscape over $\mathcal{S}$ is rugged: one width or dropout change can move macro-F1 by several points. Tournament selection, layer-block crossover, one-gene mutation, and elitism are chosen together so the search can climb local hills without abandoning an entire depth niche too early. Section 1.1 defined what is searchable; Section 1.2 defined how to search; Section 1.3 defined how to deploy the search per configuration. That three-way split is the structure of the method.

1.3.8 Chapter Summary

This standalone chapter presented GA-based DNN classifier architecture optimization in three sections. Section 1.1 encoded MLP classifiers as chromosomes with an explicit search space and parameter budget. Section 1.2 detailed population initialization, macro-F1 fitness, tournament selection, wrap-aware crossover, mutation, elitism, and termination. Section 1.3 specified configuration-specific discovery of $\mathbf{C}^*$, compared transfer and fixed baselines, and listed reproducibility requirements. The GA remains an offline design tool; online inference uses only the final decoded network.

Appendix A

Notation and Quick Reference

Table A.1: Notation.

Symbol	Meaning
$\mathbf{Z}_{fusion}$	Fused feature vector (dim. 448)
$\mathbf{C}, \mathbf{C}^*$	Chromosome; best chromosome
$\mathcal{S}, \mathcal{S} $	Search space; cardinality
$P(\mathbf{C})$	Trainable parameter count
$N, G_{\max}$	Population size; max generations
p_c, p_m, t, E	Crossover; mutation; tournament; elitism

A.1 Reproducibility Checklist

1. Genes and allowed sets match Equation (1.2).
2. Parameter count uses Equation (1.4) with budget 2×10^6 .
3. Hyperparameters match Table 1.4.
4. Crossover uses wrap-aware map (1.11).
5. Fitness is validation macro-F1; failures score 0.
6. New configurations trigger a fresh GA run unless studying transfer baselines.

A.2 Formula–Reason Index

Table A.2: Core formulas and why they were chosen.

Formula	Why chosen
Chromosome (1.1)	Readable MLP genes engineers can audit
Space (1.2)	Practical discrete ranges for 448-D features
Cardinality (1.3)	Shows enumeration is impossible
Params (1.4)	Hard compute/overfit guardrail
Fitness (1.6)	Class-balanced diagnostic objective
Crossover (1.11)	Inherit layer blocks; handle depth mismatch
Best (1.12)	Deploy a single concrete architecture
Claim (1.15)	Config-specific optimality is testable
Complexity (1.13)	Marks GA as offline design cost